

HelixMemory

HelixMemory

Un cerebro de memoria para agentes AI: cuatro motores de vanguardia, fusionados.

Resumen

HelixMemory es un Go SDK que unifica cuatro sistemas de memoria líderes (Mem0, Cognee, Letta, Graphiti) en un único motor de memoria cognitiva que los busca en paralelo y fusiona los resultados. Proporciona a las aplicaciones AI una capa de memoria duradera, deduplicada y reordenada, en lugar de cuatro capas desconectadas.

Descripción breve

HelixMemory es un Go SDK que fusiona Mem0, Cognee, Letta y Graphiti en un único motor de memoria cognitiva unificada para aplicaciones AI. Dirige las escrituras de forma inteligente, busca en todos los backends en paralelo y fusiona los resultados mediante un pipeline de tres etapas: recopilación, deduplicación y reordenación.

Descripción detallada

HelixMemory es un motor de memoria cognitiva unificada para aplicaciones AI, implementado como un Go SDK (módulo `digital.vasic.helixmemory`, Go 1.25+). Su apuesta fundacional es que ningún proyecto de memoria será jamás el mejor en todo, por lo que, en lugar de reimplementar la memoria desde cero y heredar los puntos ciegos de un único proyecto, orquesta cuatro sistemas de vanguardia y permite que cada uno desarrolle sus fortalezas: Mem0 para la extracción dinámica de hechos y la gestión de preferencias, Cognee para grafos de conocimiento semántico construidos mediante pipelines ECL, Letta para un entorno de ejecución de agentes con estado, bloques de memoria

editables y cómputo durante el “sueño”, y Graphiti para un grafo de conocimiento bitemporal que razona sobre cómo cambian los hechos a lo largo del tiempo.

El motor de fusión es lo que convierte esos cuatro almacenes independientes en un único cerebro. En la ruta de escritura, cada memoria entrante se clasifica por contenido y se dirige al backend más adecuado para almacenarla. En la ruta de lectura, una consulta se distribuye en paralelo a todos los backends y los resultados brutos fluyen hacia un pipeline de fusión de tres etapas —recopilación, deduplicación y reordenación entre fuentes— de modo que quien realiza la llamada nunca ve cuatro conjuntos de resultados ruidosos y superpuestos, sino una única respuesta limpia y ordenada. Interruptores automáticos envuelven cada backend para una degradación elegante: cuando un motor falla, su interruptor se activa y los backends restantes siguen funcionando, en lugar de arrastrar consigo toda la capa de memoria. Dado que el motor implementa una interfaz `MemoryStore` de integración directa, puede reemplazar directamente a un proveedor de memoria básico —sin necesidad de reestructurar el código que lo invoca— y las métricas Prometheus exponen los detalles internos de enrutamiento y fusión para una observabilidad completa.

HelixMemory se desarrolló como capa de memoria para HelixAgent, el conjunto más amplio de AI Helix, e incorpora la disciplina de pruebas anti-engaño de la familia al dominio de la memoria: un ejecutor de desafíos en proceso ejercita rutas de código reales en producción —enrutamiento, fusión, traductor, interruptor automático—, mientras que un envoltorio de mutación emparejado altera deliberadamente invariantes para demostrar que las pruebas fallan cuando la lógica está rota, de modo que un conjunto de pruebas en verde signifique algo.

Por qué lo creamos

Los agentes AI requieren memoria duradera y de alta calidad, pero el ecosistema está fragmentado: cada proyecto de memoria (Mem0, Cognee, Letta, Graphiti) destaca en un aspecto y flaquea en otros. HelixMemory se creó para ofrecer a HelixAgent una única capa de memoria que combine sus fortalezas sin imponer un bloqueo a ninguno de ellos.

Por qué es un cambio radical

Pone fin a la elección forzada. Cuatro sistemas de memoria que normalmente compiten por el mismo espacio se convierten en backends complementarios tras una única interfaz, de modo que una aplicación obtiene extracción dinámica de hechos, grafos de

conocimiento semántico, memoria de estado para agentes y razonamiento bitemporal *simultáneamente*, con deduplicación y reordenación entre fuentes gestionadas de forma automática. Lo que antes no era práctico —tratar “¿qué motor de memoria adoptamos?” como un falso dilema— ahora es posible: HelixMemory permite aprovechar todas sus ventajas a la vez, tras un único MemoryStore de integración directa, sin heredar los puntos ciegos de ningún motor ni comprometerse con un bloqueo.

Qué tiene de innovador

- **Fusión multibackend** (recopilar → deduplicar → reordenar entre fuentes) que devuelve un único conjunto de resultados ordenados, en lugar de encadenar al llamante a un único almacén.
- **Enrutamiento inteligente de escrituras** que clasifica cada memoria según su contenido y la envía al motor más adecuado para almacenarla, de modo que los datos correctos lleguen al almacén correcto.
- **Degradación controlada** mediante disyuntores por backend: un motor fallido se aísla, pero no es catastrófico, y los demás siguen funcionando.
- **Consolidación computacional en tiempo de inactividad** (mediante Letta) que reestructura la memoria durante los periodos de baja actividad, en lugar de hacerlo solo al consultar.
- **Verificación anti-engaño**: un ejecutor de desafíos sobre código de producción real, combinado con un envoltorio de mutación que debe fallar cuando se altera un invariante, demostrando que la compuerta de pruebas es una verificación real, no una tautología.

Principales desafíos técnicos y cómo los resolvimos

- **Unificar cuatro backends heterogéneos en un único conjunto de resultados coherente**: cada motor devuelve la memoria en su propio formato, y fusionarlos de forma ingenua genera duplicados y rankings incomparables. Solución: un motor de fusión tipado que recopila datos de todas las fuentes, elimina duplicados y reordena todo bajo un criterio común, con el invariante de conteo fusionado verificado en pruebas para evitar que la fusión pierda o duplique resultados de forma silenciosa.
- **Mantener el sistema en funcionamiento cuando un backend falla**: un motor de memoria inaccesible no debe paralizar toda la capa. Solución: disyuntores por backend que siguen un autómata de estados cerrado → abierto (tras superar

un umbral de fallos) → semiabierto (tras un tiempo de espera), aislando el backend afectado y permitiendo que los sanos sigan sirviendo hasta su recuperación.

- **Demostrar que la lógica de memoria funciona, no solo que compila:** un conjunto de pruebas en verde carece de sentido si estas no pueden fallar. Solución: un ejecutor de desafíos en proceso que opera sobre código de producción real (enrutamiento, fusión, traductor, disyuntor) y un envoltorio de mutación emparejado que altera invariantes y exige que las pruebas fallen, garantizando así que la compuerta no sea una tautología.

Pila tecnológica

- **Go (1.25+)** — el único SDK y entorno de ejecución; seleccionado porque la lectura paralela en abanico a través de cuatro backends es un problema de concurrencia, y las goroutines de Go lo hacen económico, mientras que sus tipos de interfaz ofrecen al sistema una única costura limpia (MemoryStore) en la que los llamadores pueden confiar.
- **Mem0** — el backend de extracción dinámica de hechos y gestión de preferencias; utilizado para la porción de memoria que responde a “qué prefiere realmente este usuario / qué hechos han surgido”.
- **Cognee** — el backend de grafo de conocimiento semántico construido sobre pipelines ECL; empleado para almacenar conocimiento estructurado y relacionado, en lugar de hechos planos.
- **Letta** — el backend de entorno de ejecución de agentes con estado, con bloques de memoria editables y cómputo en períodos de inactividad; utilizado cuando la memoria debe persistir como estado activo del agente y consolidarse durante los intervalos de reposo.
- **Graphiti** — el backend de grafo de conocimiento bitemporal; empleado para razonar sobre cómo cambian los hechos y las relaciones a lo largo del tiempo, no solo su valor actual.
- **PostgreSQL + Neo4j + Redis** — los almacenes de datos reales contra los que operan los backends, desplegados para pruebas de integración auténticas mediante make infra-start, de modo que el conjunto de pruebas ejercite infraestructura real en lugar de simulaciones.
- **Prometheus** — métricas y observabilidad integradas a través del pipeline de fusión, para que el enrutamiento y el comportamiento de fusión sean medibles en producción, sin cajas negras.
- **Costura de traducción i18n** — una superficie de cadenas con espacio de nombres (helixmemory_) mantenida para que cualquier capa orientada al usuario en el futuro pueda localizarse sin necesidad de adaptar el núcleo.

Estado y notas de transparencia

- **Estado: beta.** SDK funcional; desarrollado como capa de memoria para HelixAgent.
- **Licencia: por definir.** No se detectó ninguna LICENCIA mediante el GitHub API — SIN VERIFICAR / no declarada.
- El nombre para mostrar “HelixMemory” corresponde al repositorio memory. Las cifras de precisión citadas en el README son afirmaciones de proveedores externos, no mediciones de HelixMemory, y se omiten aquí.

Nivel de prioridad: Helix principal.